# I- STRUCTURES DE DONNEES ELEMENTAIRES

## I-1- LES TABLEAUX

**Objectifs**

- ✓ *Maîtriser la manipulation des tableaux à une ou à deux dimensions.*
- ✓ *Traitement des problèmes de recherche dans un tableau (Recherche séquentielle, Recherche dichotomique;*
- ✓ *Savoir les principaux algorithmes de tri d'un tableau.*

**Exemple de déclaration d'un tableau**

```
Constantes
        n = 100

Type
        Tab = Tableau[1..n] de Entier
Var
        T : Tab
```

**Exemple de déclaration d'une matrice**

```
Constantes
        n = 100
        m= 100
Type
        Mat = Tableau[1..n,1..m] de Entier
Var
        Matrice : Mat
```

### I-1-1- PROCEDURE *Remplir* (permet le remplissage d'un tableau de n entiers)

```
Procédure Remplir (var T : Tab; n: Entier)
Var
        i : Entier
Début
        Pour i de 1 à n Faire
                Ecrire("T[",i,"]="),Lire(T[i])
        Fin Pour
Fin
```

### I-1-2- PROCEDURE *Afficher* (permet l'affichage des éléments d'un tableau de n entiers)

```
Procédure Afficher (T : Tab; n: Entier)
Var
        i : Entier
Début
        Pour i de 1 à n Faire
                Ecrire(T[i], "    ")
        Fin Pour
Fin
```

**I-1-3- FONCTION *Minimum*** *(*permet de chercher le minimum dans un tableau T de n entiers)

```
Fonction Minimum(T : Tab) : Entier
Var
        min : Entier
Début
        min ← T[1]
        Pour i de 2 à n Faire
                Si ( min> T[i] ) Alors
                        min ← a
                Fin Si
        Fin Pour
Fin
```

**I-1-4- FONCTION *Recherche_seq*** *(*permet la recherche séquentielle d'un élément x dans un tableau)

**Principe de la recherche séquentielle**

Comparer x aux différents éléments du tableau jusqu'a trouver x ou atteindre la fin du tableau.

- ***Version avec Répéter… jusqu'à***

```
Fonction Recherche_seq (T:Tab; n,x:Entier) : Booléen
Var
        i, : Entier

Début
        i ← 0
        Repeter
                i ←i+1
        Jusqu'à (T[i]=x) OU (i = n)

        Si(T[i]=x) Alors
                Recherche_seq ← Vrai
        Sinon
                Recherche_seq ← Faux
        Fin Si
Fin
```

- ***Version avec Tant que… Faire***

```
Fonction Recherche_seq (T:Tab; n,x:Entier) : Booléen
Var
        i, : Entier
Début
        i ← 1
        Tant que (T[i] # x)  ET  (i <=n) Faire
                i ←i+1
        Fin Tant que

        Si(T[i]=x) Alors
                Recherche_seq ← Vrai
        Sinon
                Recherche_seq ← Faux
        Fin Si
Fin
```

**I-1-5- FONCTION *Recherche_dich*** (permet la recherche dichotomique d'un élément x dans un tableau)

**Principe de la recherche dichotomique**

Le but de la recherche dichotomique est de diviser l'intervalle de recherche par 2 à chaque itération. Pour cela, on procédé de la façon suivante :

Soient premier et dernier les extrémités gauche et droite de l'intervalle dans lequel on cherche la valeur x, on calcule m, l'indice de l'élément médian :

- m = (premier + dernier) Div 2

Il y a trois cas possibles :

- x < T[m] : l'élément x, s'il existe, se trouve dans l'intervalle [premier..m-1].
- x > T[m] : l'élément x, s'il existe, se trouve dans l'intervalle [m+1...dernier].
- x = T[m] : l'élément de valeur x est trouve, la recherche est terminée.

La recherche dichotomique consiste a iterer ce processus jusqu'a ce que l'on trouve x ou que l'intervalle de recherche soit vide.

```
Fonction Recherche_dich(T:Tab; n,x:Entier) : Booléen
Var
        premier, m, dernier : Entier
        trouve : Booléen
Début
        premier ← 1
        dernier ← n
        trouve ← Faux
        Répéter
                m←(premier + dernier) Div 2
                                                    (1)
        Si ( x < T[m] ) Alors
            dernier ← m – 1
        Sinon Si ( x > T[m] ) Alors
            premier ← m + 1
        Sinon
            trouve ← Vrai
        Fin Si
    Jusqu'à (trouve=vrai) OU (premier>dernier)
    Recherche_dich ← trouve
Fin
                                                    (2)
```

**I-1-6- PROCEDURE *Tri_Bulles***

**Principe :**

On répète le traitement suivant :

Parcourir les éléments du tableau de 1 a (n-1); si l'élément i est supérieur a l'élément (i+1), alors on permute.

Le programme s'arrête lorsqu'aucune permutation n'est réalisable.

```
Procédure Tri_Bulles( var T: Tab; n: Entier)
Var
        i, aux : Entier
        échange : Booléen
Début
        Répéter
                échange ← Faux
                Pour i de 1 à (n-1) Faire
                        Si(T[i] > T[i+1]) Alors
                                aux ← T[i]
                                T[i] ← T[i+1]
                                T[i+1] ← aux
                                échange ←Vrai
                        Fin Si
                Fin Pour
        Jusqu'à (échange = Faux)
Fin
```

**Optimisation de la procédure du Tri à bulle**

```
Procédure Tri_Bulles( var T: Tab; n: Entier)
Var
        i, aux : Entier
        échange : Booléen


Début
        Répéter
                échange ← Faux
                Pour i de 1 à (n-1) Faire
                        Si(T[i] > T[i+1]) Alors
                                aux ← T[i]
                                T[i] ← T[i+1]
                                T[i+1] ← aux
                                échange ←Vrai
                        Fin Si
                Fin Pour
                n←n-1
        Jusqu'à (échange = Faux) OU (n = 1)
Fin
```

**Trace d'exécution**

| | | | | | |
|---|---|---|---|---|---|
| *Tableau initial* | 6 | 4 | 2 | 1 | 4 |
| *Après la 1ère itération* | 4 | 2 | 1 | 4 | 6 |
| *Après la 2ème itération* | 2 | 1 | 4 | 4 | 6 |
| *Après la 3ème itération* | 1 | 2 | 4 | 4 | 6 |
| *Après la 4ème itération* | 1 | 2 | 4 | 4 | 6 |

### I-1-7- PROCEDURE *Tri_Selection*

**Principe :**

Cette méthode consiste :

– chercher l'indice du plus petit élément du tableau T[1..n] et permuter l'élément correspondant avec l'élément d'indice 1;

– chercher l'indice du plus petit élément du tableau T[2..n] et permuter l'élément correspondant avec l'élément d'indice 2;

– …

– chercher l'indice du plus petit élément du tableau T[n-1..n] et permuter l'élément correspondant avec l'élément d'indice (n-1);

```
Procédure Tri_Selection( var T: Tab; n: Entier)
Var
        i,j,aux,indmin: Entier
Début
        Pour i de 1 à (n-1) Faire
                indmin ← i
                Pour j de (i+1) à n Faire
                        Si(T[j] < T[indmin]) Alors
                                indmin ← j
                        Fin Si
                Fin Pour
                aux ← T[i]
                T[i] ← T[indmin]
                T[indmin]←aux
        Fin Pour
Fin
```

**Trace d'exécution**

| | | | | | |
|---|---|---|---|---|---|
| *Tableau initial* | 6 | 4 | 2 | 1 | 4 |
| *Après la 1ère itération* | 1 | 4 | 2 | 6 | 4 |
| *Après la 2ème itération* | 1 | 2 | 4 | 6 | 4 |
| *Après la 3ème itération* | 1 | 2 | 4 | 6 | 4 |
| *Après la 4ème itération* | 1 | 2 | 4 | 4 | 6 |

### I-1-8- PROCEDURE *Tri_Insertion*

**Principe :**

Cette méthode consiste a prendre les éléments de la liste un par un et insérer chacun dans sa bonne place de façon que les éléments traites forment une sous-liste triée.

Pour ce faire, on procède de la façon suivante :

– comparer et permuter si nécessaire T[1] et T[2] de façon a placer le plus petit dans la case d'indice 1

– comparer et permuter si nécessaire l'élément T[3] avec ceux qui le précédent

dans l'ordre (T[2] puis T[1]) afin de former une sous-liste triée T[1..3]

– …

– comparer et permuter si nécessaire l'élément T[n] avec ceux qui le précédent dans l'ordre (T[n-1] , T[n-2],…) afin d'obtenir un tableau trie.

```
Procédure Tri_Insertion( var T: Tab; n: Entier)
Var
        i,j,x,pos: Entier
Début
        Pour i de 2 à n Faire
                pos ← i -1
                Tant que (pos >=1) ET (T[pos] >T[i]) Faire
                        pos ← pos - 1
                Fin Tant que
                pos ← pos + 1
                x ← T[i]
                Pour j de (i-1) à pos [pas = -1] Faire
                        T[j+1] ← T[j]
                Fin Pour
                T[pos]←x
        Fin Pour
Fin
```

**Trace d'exécution**

| | | | | | |
|---|---|---|---|---|---|
| *Tableau initial* | 6 | 4 | 2 | 1 | 4 |
| *Après la 1ère itération* | 4 | 6 | 2 | 6 | 4 |
| *Après la 2ème itération* | 2 | 4 | 6 | 6 | 4 |
| *Après la 3ème itération* | 1 | 2 | 4 | 6 | 4 |
| *Après la 4ème itération* | 1 | 2 | 4 | 4 | 6 |

**I-1-8- PROCEDURE *Fusion*** *(*permet de fusionner deux tableaux tries A et B contenant respectivement n et m éléments)

```
Procédure Fusion (A : Tab; B:Tab; var C:Tab; n, m:Entier; var k :Entier)
Var
        i,j : Entier
Début
        Tant que (i <=n)  ET  (j <= m) Faire
                Si(A[i] <=B[j]) Alors
                        C[k] ← A[i]
                        i ← i + 1
                        k ← k +1
                Fin Si
                Si (B[j] <= A[i]) Alors
                        C[k] ← B[j]
                        j ← j + 1
                        k ← k +1
                Fin Si
        Fin Tant que

        Tant que (i <=n) Faire
                C[k] ← A[i]
                i ← i + 1
                k ← k +1
        Fin Tant que
        Tant que (j <=m) Faire
                C[k] ← B[j]
                j ← j + 1
                k ← k +1
        Fin Tant que
Fin
```

*On a l'élément de A et plus petit que B alors on ajoute A dans C*

*On a l'élément de A et plus grand que B alors on ajoute B dans C*

*Si on a encore des éléments dans A alors on les rajoute successivement dans C*

*Si on a encore des éléments dans B alors on les rajoute successivement dans C*

**I-1-9- PROCEDURE *Remplir_Matrice*** *(*permet de remplir une matrice)

```
Remplissage d'une matrice
    Procédure Remplir (var matrice : Mat; n,m: Entier)
    Var
            i,j : Entier
    Début
            Pour i de 1 à n Faire
                    Pour j de 1 à m Faire
                            Ecrire("Entrer un entier : "),Lire(T[i,j])
                    Fin Pour
            Fin Pour
    Fin
```

### I-1-10- PROCEDURE *Afficher_Matrice* (permet de d'afficher une matrice)

```
Affichage d'une matrice
    Procédure Afficher (matrice : Mat; n,m: Entier)
    Var
            i,j : Entier
    Début
            Pour i de 1 à n Faire
                    Pour j de 1 à m Faire
                            Ecrire(T[i,j])
                    Fin Pour
            Fin Pour
    Fin
```

### I-1-11- FONCTION *Somme_Matrice* (permet de renvoyer la somme de deux matrices M1 et M2)

**Principe**:
Soient M1 et M2 deux matrices à n ligne et m colonnes.
M3 = M1 + M2

**Exemple**

**M1=**

| | | |
|---|---|---|
| 1 | 2 | 3 |
| 4 | 5 | 6 |

**M2=**

| | | |
|---|---|---|
| 2 | 5 | 3 |
| 3 | 0 | 1 |

**M3=**

| | | |
|---|---|---|
| 3 | 7 | 6 |
| 7 | 5 | 7 |

```
Procédure SomMat (M1,M2 : Mat ; var M3 : Mat; n,m: Entier)
Var
        i,j : Entier
Début
        Pour i de 1 à n Faire
                Pour j de 1 à m Faire
                        M3[i,j] ← M1[i,j] + M2[i,j]
                Fin Pour
        Fin Pour
Fin
```

**I-1-12- FONCTION *Produit_Matrice*** *(*permet de renvoyer le produit de deux matrices M1 et M2)

**Principe**:
Soient M1 et M2 deux matrices à n ligne et m colonnes.
M3 = M1 * M2
$M3_{i,j} = M1_{i,1} * M2_{1,j} + M1_{i,2} * M2_{2,j} + \ldots + M1_{i,n} * M2_{m,j}$

**Exemple**

**M1=**

| | | |
|---|---|---|
| 1 | 2 | 3 |
| 4 | 0 | 5 |

**M2=**

| | |
|---|---|
| 2 | 1 |
| 3 | 0 |
| 1 | 4 |

**M3=**

| | |
|---|---|
| 11 | 13 |
| 13 | 24 |

```
Procédure ProdMat (M1,M2 : Mat ; var M3 : Mat; n,m: Entier; var k : Entier)
Var
        i,j : Entier
Début
        Pour i de 1 à n Faire
                Pour j de 1 à m Faire
                        M3[i,j] ← 0
                        Pour k de 1 à m Faire
                                M3[i,j] ← M3[i,j] + M1[i,k] * M2[k,j]
                        Fin Pour
                Fin Pour
        Fin Pour
Fin
```

## I-2- LES LISTES CHAINEES

**Objectifs :**

- ✓ *Savoir déclarer, construire des listes chainées.*
- ✓ *Manipuler, traiter et apprendre à utiliser des listes chainées.*
- ✓ *Distinguer entre les différents types de listes chainées.*

### DEFINITIONS

***1)*** *Un* ***pointeur*** *P est une variable statique dont les valeurs sont des adresses.*

***2)*** *La différence entre une structure de données statique et une structure de données dynamique est la taille, la première est définie au début en spécifiant la taille qui est fixe, par contre pour la deuxième la taille est variable et qui est basée sur l'allocation dynamique.*

### DECLARATION ET UTILISATION

*P : ^Entier //déclare un pointeur vers une donnée de type entier*

*Allouer(P) // alloue de l'espace mémoire nécessaire à une donné de type entier et place son adresse dans P*

*P^=5 // affecte la donnée 5 à l'espace mémoire alloué*

*Libérer(P) // libère l'espace mémoire dont l'adresse se trouve dans P*

### EXEMPLE

```
Type
   Liste : ^cellule
   Cellule : Enregistrement
      val  : Entier
      suiv : Liste
   Fin Cellule
```

```
Var
   L : Liste
```

**I-2-1- PROCEDURE CréerListe** (Permet de créer une liste chainée de n entiers)

```
Procédure CréerListe(n : Entier ; var L : Liste)
Var
Tete, p : Liste
   i : Entier
Début
     Allouer(Tete)
     Ecrire("Entrer élément Tête :")
     Lire(Tete^.val)
     Tete^.suiv ← nil
     L ← Tete

     Pour i de 2 à n faire
          Allouer(p)
          Ecrire("Entrer élément n° :",i)
          Lire(p^.val)
          p^.suiv ← nil
          L^.suiv ← p
          L ← p
     Fin Pour
     L ← Tete //se pointer sur la tête de la liste
Fin
```

**I-2-2- PROCEDURE Affichage**(Permet d'afficher les éléments de la liste)

```
Procédure AffichageIter(L : Liste)
Var
    p : Liste
Début
     p ← L
     Tant que p ≠ nil Faire
          Ecrire(p^.val)
          p ← p^.suiv
     Fin Tant que
Fin
```

**I-2-3- FONCTION Recherche** (Permet de rechercher un element x dans la liste)

*Version itérative*

```
Fonction Recherche(x : Entier ; L : Liste) : Booléen
Var
   p : Liste
Début
      p ← L
      Tant que ((p ≠ nil) ET (p^.val ≠ x))  Faire
            p ← p^.suiv
      Fin Tant que
      Si p = nil Alors
            Recherche← Faux
      Sinon
            Recherche← Vrai
      Fin Si
Fin
```

*Version récursive*

```
Fonction Recherche(x : Entier ; L : Liste) : Booléen
Début
      Si L = nil Alors
            Recherche← Faux
      Sinon
            Si L^.val = x  Alors
                  Recherche ← Vrai
            Sinon
                  Recherche ← Recherche(x, L^.suiv)
            Fin Si
      Fin Si
Fin
```

**I-2-4- PROCEDURE AjouterTete** (Permet d'ajouter une tete de la liste)

```
Procédure AjouterTete(x : Entier ; var L : Liste)
Var
      Tete : Liste
Début
      Allouer(Tete)
      Tete^.val ← x
      Tete^.suiv ← L
      L←Tete
Fin
```

**I-2-5- PROCEDURE Supprimer** (Permet de supprimer un element de la liste)

*Version itérative*

```
Procédure Supprimer(x : Entier ; var L : Liste)
Var
  P,Q : Liste
Début
      Si L = nil Alors
            Ecrire("Liste vide, impossible de supprimer ",x)
      Sinon
            SiL^.val = x Alors
                  P ← L //mémorisation de l'élément à supprimer
                  L ← L^.suiv
            Sinon
                  P ← L^.suiv
                  Tant que ((P ≠ nil) ET (P^.val ≠ x)) Faire
                        Q ← P
                        P ← P^.suiv
                  Fin Tant que
                  Si P ≠ nil Alors
                        Q^.suiv ← P^.suiv
                        Libérer(P)
                  Fin Si
            Fin Si
      Fin Si
Fin
```

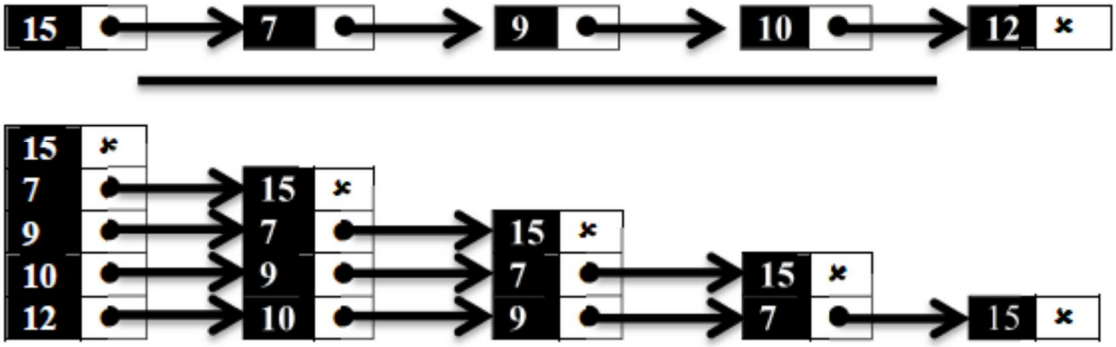
*Version récursive*

```
Procédure Supprimer(x : Entier ; var L : Liste)
Var
P : Liste
Début
      Si L ≠ nilAlors
            Si L^.val = x Alors
                  P ←L
                  L ← L^.suiv
                  Libérer(P)
            Sinon
                  Supprimer(x, L^.Suiv)
            FinSi
      FinSi
Fin
```

**I-2-6- PROCEDURE Inverse** (Permet d'inverser une liste)

```
Procédure Inverse (var L : Liste)
Var
   P,Q : Liste
Début
      P ←nil
      Tant que L ≠ nil Faire
            Q ← L^.suiv
            L^.suiv ← P
            P ← L
            L ← Q
      Fin Tant que
Fin
```

**Trace d'exécution de l'exemple ci-dessous**



**I-2-7- PROCEDURE TriBulle** (Permet de trier une liste chainee selon le principe de tri a bulle)

```
Procédure TriBulle(var L : Liste)
Var
   P, Q : Liste
Temp : Entier
Début
     P ← L
     Q ← L^.suiv
     Répéter
          rep← faux
          P ← L          // pointeur sr le premier élément
          Q ← L^.suiv    // pointeur sur le deuxième élément
          Tant que Q ≠ nil Faire
               Si P^.val>Q^.val Alors
                    Temp ← P^.val
                    P^.val ← Q^.val
                    Q^.val ← Temp
                    rep ← vrai
               Fin Si
          Fin tant que
     Jusqu'à rep = faux
Fin
```

**Permutation**

**I-2-8- PROCEDURE Concatener** (Permet de concatener deux listes chainees L1 et L2 d'entiers dans une troisieme liste L3)

```
Procédure Concatener(L1,L2 : Liste ; var L3 : Liste)
Début
    Si L1 = nil ET L2 = nil Alors
        L3 ←nil
    Sinon Si L1 ≠ nil ET L2 = nil Alors
        L3 ← L1
    Sinon Si L1 = nil ET L2 ≠ nil Alors
        L3 ← L2
    Sinon
        //affectation de la première liste L1
        L3 ← L1
        //se pointer sur le dernier élément de L3
        Tant que L3^.suiv ≠ nil Faire
            L3 ← L3^.suiv
        Fin Tant que
        //affectation de la liste L2 à la fin de L3
        L3^.suiv ← L2
        //se pointer sur la tête de la liste
        L3 ← L1
    Fin Si
Fin
```

**I-2-8- PROCEDURE Fusion** (Permet de fusionner deux listes triée L1 et L2 dans la liste L1)

```
Procédure Fusion(var L1 :Liste ; L2 : Liste)
Var
Tete, P, Q : Liste
Début
      Si L1 = nil Alors   //Si L1 et vide
            L1 ← L2
      Sinon
            Si L2 ≠ nilAlors
                  //Fixation de la position de la tête de la liste
                  Si L1^.val ≤ L2^.val Alors
                        Tete ← L1
                  Sinon
                        Tete ← L2
                  Fin Si
                  Q ←Tete  //mémorisation de l'élément précédent
                  Tant que L1 ≠ nil ET L2 ≠ nil Faire
                        Si L1^.val > L2^.val Alors
                              P ← L2
                              L2 ← L2^.suiv
                              Q^.suiv ← P
                              P^.suiv ← L1
                              Q ← P
                        Sinon Si L1^.val > L2^.val Alors
                              P ← L2
                              L2←L2^.suiv
                              Q ← L1
                              L1←L1^.suiv
                              Q^.suiv← P
                              P^.suiv← L1
                        Sinon
                              Q ← L1  //mémorisation de l'élément précédent
                              L1 ← L1^.suiv
                        Fin Si
                  Fin Tant que
                  L1 ←Tete      //pointer L1 à la tête de la liste
            Fin Si
      Fin Si
Fin
```

**I-2-9- PROCEDURE SupprimeDoublons**(Permet de Supprimer les doublons dans une liste chainee)

```
Procédure SupprimeDoublons(var L :Liste)
Var
Q , P : Liste
Début
      Q ← L
      Si L ≠ nilAlors
            Tant que Q^.suiv ≠ nilFaire
                  P ← Q
                  Q ← Q^.suiv
                  Si Q^.val = P^.valAlors
                        //on supprime de L l'élément pointé par Q.
                        Q^.suiv ← P^.suiv
                        Libérer(P)
                  Fin Si
            Fin Tant que
      Fin Si
Fin
```

```
Procédure SupprimeDoublons(var L :Liste)
Var
    Q : Liste
    Vpred : caractère
Début
      Q ← L
      Si L ≠ nilAlors
            Tant que Q^.suiv ≠ nilFaire
                  Vpred←Q^.val
                  Q ← Q^.suiv
                  Si Q^.val = Vpred Alors
                        Supprimer(Vpred,Q)
                  Fin Si
            Fin Tant que
      Fin Si
Fin
```

**Cette procédure se charge de parcourir la liste Q en supprimant toutes les valeurs de Vpred**

**I-2-10- FONCTION Palindrome** (Permet de vérifier si une liste est palindrome ou non)

```
Fonction Palindrome(L : Liste) :Booléen
Var
      Stop : Booléen
      Deb, Fin, Q : Liste
Début
      Deb← L            // Pointeur sur le premier élément
      Fin←nil           // Pointeur sur le dernier élément
      Stop ← faux       // Signale l'interruption du traitement
      Tant que Deb^.suiv ≠ FinETNON(Stop) Faire
            // Récupérer le pointeur sur le dernier élément de la liste
            Q←P
            Tant que Q^.suiv ≠ FinFaire
                  Q ← Q^.suiv
            Fin Tant que

            Si P^.val = Q^.val Alors
                  Fin← Q  //se pointer sur le dernier élément de la liste L
            Sinon
                  Stop ← vrai   // arrêt du traitement
            Fin Si
            Si Deb ≠ Fin Alors
               Deb ← Deb ^.suiv
            Fin Si
      Fin Tant que
      Palindrome← NON(Stop)
Fin
```

## I-3- LES PILES

### I-3-1- DEFINITION

Une pile est une structure de données dynamique (liste chaînée) dont l'insertion et la suppression d'un élément se font toujours en tête de liste.

On peut résumer les contraintes d'accès par le principe « dernier arrivé, premier sorti » qui se traduit en anglais par : Last In First Out.

### I-3-2- CREATION DE LA STRUCTURE D'UNE PILE

```
Type
      Pile : ^cellule
      cellule : Enregistrement
            val  : Entier
            suiv : Pile
      Fin cellule
```

### I-3-3- DECLARATION D'UNE VARIABLE DE TYPE PILE

```
Var
      P : Pile
```

### I-3-4- PROCEDURE *InitialiserPile(var P : Pile)* *(permet de créer une pile vide)*

```
Procédure InitialiserPile(var P : Pile)
Début
      P ← nil
Fin
```

### I-3-5- FONCTION *EstPileVide(P: Pile)* *(permet de vérifier si une pile est vide)*

```
Fonction EstPileVide(P : Pile) : Booléen
Début
      Si P = nil Alors
            EstPileVide ← vrai
      Sinon
            EstPileVide ← faux
      Fin Si
Fin
```

**I-3-6- PROCEDURE** ***Empiler(x : Entier ; var P : Pile)*** *(permet d'ajouter l'élément x au sommet de la pile)*

```
Procédure Empiler(x : Entier ; var P : Pile)
Var
      Nv : Pile
Début
      Allouer(Nv)
      Nv^.val ← x
      Nv^.suiv ← P
      P ← Nv
Fin
```

**I-3-7- PROCEDURE** ***Dépiler (var x : Entier ; var P : Pile)*** *(permet de supprimer le sommet de la pile et de le mettre la valeur dans la variable x)*

```
Procédure Dépiler(var x : Entier ; var P : Pile)
Var
      S : Pile
Début
      Si NON(EstPileVide(P)) Alors
            S ← P
            x ← P^.val
            P ← P^.suiv
            Libérer(S)
      Fin Si
Fin
```

## I-4- LES FILES

### I-4-1- DEFINITION

Une file est une structure de données dynamique (liste chaînée) dont les contraintes d'accès sont définies comme suit :

- On ne peut ajouter un élément qu'en dernier rang de la suite.
- On ne peut supprimer que le premier élément.

On peut résumer les contraintes d'accès par le principe « premier arrivé, premier sorti » qui se traduit en anglais par : Fast In First Out.

### I-4-2- CREATION DE LA STRUCTURE D'UNE FILE

```
Type
      Liste : ^cellule
      cellule : Enregistrement
      |     val  : Entier
      |     suiv : Liste
      Fin cellule

      File : Enregistrement
      |     Tête  : Liste
      |     Queue : Liste
      Fin File
```

### I-4-3- DECLARATION D'UNE VARIABLE DE TYPE FILE

```
Var
      F : File
```

### I-4-4- PROCEDURE *InitialiserFile(var F : File)* *(permet de créer une file vide)*

```
Procédure InitialiserFile(var F : File)
Début
|     F.Tête  ← nil
|     F.Queue ← nil
Fin
```

### I-4-5- FONCTION *EstFileVide(F : File)* *(permet de vérifier si une file est vide)*

```
Fonction EstFileVide(F : File) : Booléen
Début
|     Si F.Tête = nil Alors
|     |     EstPileVide ← vrai
|     Sinon
|     |     EstPileVide ← faux
|     Fin Si
Fin
```

**I-4-6- PROCEDURE** ***Enfiler(x : Entier ; var F : File)*** *(permet d'ajouter l'élément x au sommet de la file)*

```
Fonction Enfiler(x : Entier ; var F : File)
var
      Nv : Liste
Début
      Allouer(Nv)
      Nv^.val ← x
      Nv^.suiv ← nil
      Si F.Queue ≠ nil Alors
            F.Queue^.suiv ← Nv
      Sinon
            F.Tête ← Nv
      Fin Si
      F.Queue ← Nv
Fin
```

**I-3-7- PROCEDURE** ***Défiler(var x : Entier ; var F : File)*** *(permet de supprimer le sommet de la file et de le mettre la valeur dans la variable x)*

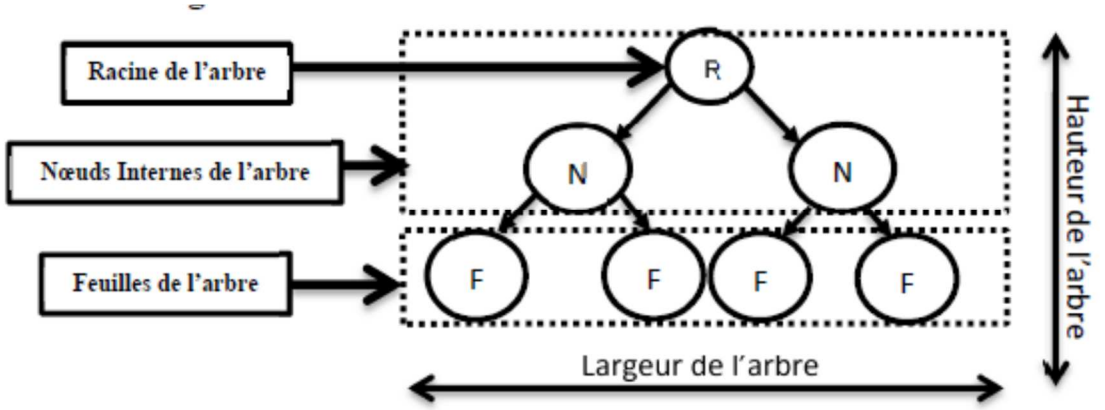
```
Procédure Défiler(var B : Arbre ; var F : File)
var
      P : Liste
Début
      Si F.Tête ≠ nil Alors
            P ← F.Tête
            B ← P^.val
            F.Tête ← F.Tête^.suiv
            Libérer(P)
      Fin Si
      Si F.Tête = nil Alors
            F.Queue ← nil
      Fin Si
Fin
```

## II- EVALUATION DES ALGORITHMES

## III- ARBRE BINAIRE DE RECHERCHE

### III-1 DEFINITION

Un **arbre binaire** B est un ensemble de nœuds qui est soit vide, soit compose d'une racine et de deux arbres binaires disjoints **appelés sous-arbre droit** et **sous-arbre gauche**.




- Un **nœud interne** est un sommet qui a au moins un fils (gauche ou droit ou les deux).
- Une **feuille** est un sommet qui n'a pas de fils.
- La **hauteur d'un sommet** x est la longueur (en nombre d'arcs) du plus long chemin de x à une feuille.
- La **hauteur d'un arbre** est égale à la hauteur de la racine.

### III-2 STRUCTURE D'UN ARBRE BINAIRE

```
Type
      Arbre : ^Nœud
      Nœud : Enregistrement
            Val : Entier
            FilsG : Arbre
            FilsD : Arbre
      Fin Nœud
```

### III-3 DECLARATION

```
Var
      A : Arbre
```

### III-4 PROCEDURE CréerElement (Permet de créer un élément X dans l'arbre)

```
Procédure CréerElement(x : Entier ; var B : Arbre)
Début
      Si B = nilAlors
            Allouer(B)
            B^.val← x
            B^.FilsG←nil
            B^.FilsD←nil
      Sinon
            Si B^.val> x Alors
                  CréerElement(x, B^.FilsG)
            Fin Si
            Si B^.val< x Alors
                  CréerElement(x, B^.FilsD)
            Fin Si
      Fin Si
Fin
```

**III-5 FONCTION EstVide** (Permet de dire si l'arbre est vide ou non)

```
Fonction EstVide(B :Arbre) : Booléen
Début
      Si B = nilAlors
            EstVide← faux
      Sinon
            EstVide← vrai
      Fin Si
Fin
```

**III-6 FONCTION EstUneFeuille** (Permet de dire si le sommet est une feuille ou non)

```
Fonction EstUnFeuille(B :Arbre) : Booléen
Début
      Si B^.FilsG = nil ET B^.FilsD = nil Alors
            EstUnFeuille ← vrai
      Sinon
            EstUnFeuille ← faux
      Fin Si
Fin
```

**III-7 FONCTION Recherche** (Permet de rechercher un élément dans l'arbre)

```
Fonction Recherche(x : Entier ; B : Arbre) : Booléen
Début
    Si B=nil Alors
        Recherche ← faux
    Sinon
        Si B^.val = x Alors
            Recherche ← vrai
        Sinon Si B^.val> x Alors
            Recherche ← Recherche(x, B^.FilsG)
        Sinon
            Recherche ← Recherche(x, B^.FilsD)
        Fin Si
    Fin Si
Fin
```

**III-8 PROCEDURE Supprimer** (Permet de supprimer un élément de l'arbre)

Le principe de suppression doit obéir aux constations suivantes :

La suppression commence par la recherche de l'élément. Une fois trouve ce dernier :

- Si c'est une feuille, on la vire sans problèmes
- Si c'est un sommet qui n'a qu'un fils, on le remplace par ce fils
- Si c'est un sommet qui a deux fils, on a deux solutions :

**1.** le remplacer par le sommet de plus grande valeur dans le sous arbre gauche.

**2.** le remplacer par le sommet de plus petite valeur dans le sous arbre droit.

- Pour simplifier le travail nous allons commencer par écrire deux fonctions : la première renvoie l'élément qui a la plus grande valeur dans le sous arbre gauche ; la deuxième renvoie l'élément qui a la plus petite valeur dans le sous arbre droit.

```
Fonction PlusPetitSousArbreDroit(B : Arbre)
Début
    Si B^.FilsG ≠ nil Alors
        PlusPetitSousArbreDroit ← PlusPetitSousArbreDroit(B^.FilsG)
    Sinon
        PlusPetitSousArbreDroit ← B
    Fin Si
Fin
```

```
Fonction PlusGrandSousArbreGauche(B : Arbre)
Début
    Si B^.FilsD ≠ nil Alors
        PlusGrandSousArbreGauche ← PlusGrandSousArbreGauche(B^.FilsD)
    Sinon
        PlusGrandSousArbreGauche ← B
    Fin Si
Fin
```

```
Procédure Supprimer(x : Entier ; var B : Arbre)
Var
      P, Q : Arbre
Début
      Si B = nil Alors
            Ecrire ("Arbre vide ", x, " est introuvable")
      Sinon
        Si B^.val = x Alors
            Si B^.FilsG = nil ET B^.FilsD = nil Alors
             // Si c'est une feuille
                Libérer(B)
            Sinon Si B^.FilsG ≠ nil ET B^.FilsD = nil Alors
             // Si le sommet admet un sous arbre gauche
                B←B^.FilsG
            Sinon Si B^.FilsG = nil ET B^.FilsD ≠ nil Alors
             // Si le sommet admet un sous arbre gauche
                B←B^.FilsD
            Sinon Si B^.FilsG ≠ nil ET B^.FilsD ≠ nil Alors
                  // Si le sommet admet deux fils
                  // On cherche le plus petit ou le plus grand
                  P ← PlusPetitSousArbreDroit(B^.FilsD)
                  // ou aussi on peut chercher le plus grand
                  // P ← PlusGrandSousArbreGauche(B^.FilsG)
                  Q ← P
                  Q^.FilsG ← B^.FilsG
                  Q^.FilsD ← B^.FilsD
                  Libérer(P)
                  B ← Q
            Fin Si
        Sinon Si B^.val > x Alors
           Supprimer(x, B^.FilsG)
        Sinon
           Supprimer(x, B^.FilsD)
        Fin Si
      Fin Si
Fin
```